SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED
CO4: Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)
Activity Tool: Code Challenge (High)
Assessment Tool Weightage: 35%

QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.	BL3 – Apply	5
2	Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.	BL3 – Apply	5
3	Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.	BL4 – Analyze	5
4	Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.	BL3 – Apply	5
5	Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.	BL3 – Apply	5
6	Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.	BL4 – Analyze	5
7	Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.	BL3 – Apply	5
8	Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.	BL4 – Analyze	5
9	Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.	BL4 – Analyze	5
10	Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.	BL4 – Analyze	5

Code of Conduct:
I SANTHOSH.E (19511470) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the student:
E.Santhosh

1.Heap File Organization – Insert, Delete and Sequential Scan
A heap file stores records without any particular ordering. New records can be inserted at the end, while searching requires a sequential scan.

C Program:

```
#include <stdio.h>

#include <string.h>

struct Student {

   int id;

   char name[30];

};

struct Student file[100];

int n = 0;

void insert(int id, char name[]) {

   file[n].id = id;

   strcpy(file[n].name, name);

   n++;

}

void deleteRecord(int id) {

   for (int i = 0; i < n; i++) {

      if (file[i].id == id) {

         for (int j = i; j < n - 1; j++)

            file[j] = file[j + 1];

         n--;

         printf("Record deleted\n");
```

```
            return;
        }
    }
    printf("Record not found\n");
}
void display() {
    printf("\nHeap File Records:\n");
    for (int i = 0; i < n; i++)
        printf("%d  %s\n", file[i].id, file[i].name);
}
int main() {
    insert(101, "Arun");
    insert(102, "Bala");
    insert(103, "Cathy");
    display();
    deleteRecord(102);
    display();
    return 0;
}
```

Key points:

• Records are stored in insertion order.

• Insertion is simple and fast.

• Deletion requires locating the record.

• Sequential scan checks records one by one.

Q2. Static Hashing for a Student Database with 500 Records

Static hashing maps a key to a fixed bucket using a hash function.

Assume:

• Number of records = 500

• Number of buckets = 10

• Hash function: h(key) = key % 10

C Program:

```
#include <stdio.h>
#define BUCKETS 10
int table[BUCKETS][100];
int count[BUCKETS] = {0};
void insert(int key) {
    int index = key % BUCKETS;
    table[index][count[index]++] = key;
}
void display() {
    for (int i = 0; i < BUCKETS; i++) {
        printf("Bucket %d: ", i);
        for (int j = 0; j < count[i]; j++)
            printf("%d ", table[i][j]);
        printf("\n");
    }
}
int main() {
    for (int i = 1; i <= 500; i++)
        insert(i);
    display();
    return 0;
}
```

Example:

21 % 10 = 1

31 % 10 = 1

41 % 10 = 1

These values cause a collision. Collision can be handled using chaining.

Average search complexity: O(1)

Worst-case search complexity: O(n)

Q3. Sorted File Organization and Binary Search
In a sorted file, records are stored according to the primary key. Binary search can retrieve a record efficiently.

C Program:

```
#include <stdio.h>

struct Student {
   int id;
   char name[20];
};

int binarySearch(struct Student a[], int n, int key) {
   int low = 0, high = n - 1;
   while (low <= high) {
      int mid = (low + high) / 2;
      if (a[mid].id == key)
         return mid;
      else if (a[mid].id < key)
         low = mid + 1;
      else
         high = mid - 1;
   }
   return -1;
```

```
}

int main() {
    struct Student s[] = {
        {101, "Arun"},
        {105, "Bala"},
        {110, "Cathy"},
        {115, "David"},
        {120, "Esha"}
    };
    int n = 5, key = 110;
    int pos = binarySearch(s, n, key);
    if (pos != -1)
        printf("Found: %d %s\n", s[pos].id, s[pos].name);
    else
        printf("Record not found\n");
    return 0;
}
```

For key 110:

Low = 0, High = 4, Mid = 2, and a[2] = 110, so the record is found.

Sequential search: O(n)

Binary search: O(log n)

Conclusion: Sorted file organization combined with binary search provides efficient primary-key retrieval.

Q4. B-Tree of Order 4 – Insert 10, 20, 5, 6, 12, 30, 7, 17
For a B-Tree of order 4:

• Maximum children = 4

• Maximum keys = 3

• When a node gets 4 keys, it is split.

Insertion sequence:

10, 20, 5, 6, 12, 30, 7, 17

Step 1: [10]

Step 2: [10 20]

Step 3: [5 10 20]

Step 4: Insert 6 → [5 6 10 20]

Split and promote 10:

```
      [10]
     /    \
  [5 6]   [20]
```

Step 5: Insert 12:

```
      [10]
     /    \
  [5 6]  [12 20]
```

Step 6: Insert 30:

```
      [10]
     /    \
  [5 6]  [12 20 30]
```

Step 7: Insert 7:

```
      [10]
     /    \
 [5 6 7]  [12 20 30]
```

Step 8: Insert 17 → right node becomes [12 17 20 30].

Split and promote 20.

Final B-Tree:

```
        [10 | 20]
```

/ | \

[5 6 7] [12 17] [30]

The B-Tree supports insertion, searching, deletion and traversal while keeping its height small.

Q5. B+ Tree with Leaf-Level Linking for Range Queries
A B+ Tree stores actual data entries at the leaf level. Leaf nodes are linked sequentially.

Example keys:

5, 10, 15, 20, 25, 30, 35, 40

Structure:

[20 | 30]

/ | \

[5 10 15] [20 25] [30 35 40]

↓ ↓ ↓

------→------→--------

Range query: Keys between 15 and 35.

Start at 15 and follow leaf links:

15 → 20 → 25 → 30 → 35

Advantages:

• Efficient search.

• Efficient range queries.

• Leaf-level linking supports sequential traversal.

A simple C structure is:

```
struct Node {
   int key[4];
   struct Node *child[5];
   struct Node *next;
   int leaf;
};
```

Conclusion: B+ Trees are highly suitable for database indexes because they support fast search and efficient range retrieval.

Q6. Dense Primary Index on a Sorted File
A dense primary index contains an index entry for every record in the sorted file.

Sorted file:

Record Position	Student ID
1	101
2	105
3	110
4	115
5	120

Dense Index:

Index Key	Address
101	1
105	2
110	3
115	4
120	5

C Program:

```
#include <stdio.h>

struct Index {

   int key;

   int address;

};

int main() {

   struct Index index[] = {

      {101, 1},
```

```
        {105, 2},
        {110, 3},
        {115, 4},
        {120, 5}
    };
    int n = 5, key = 115;
    for (int i = 0; i < n; i++) {
        if (index[i].key == key) {
            printf("Key found\n");
            printf("Record address = %d\n", index[i].address);
            return 0;
        }
    }
    printf("Key not found\n");
    return 0;
}
```

Advantages:

• Fast retrieval.

• One index entry per record.

• Simple implementation.

Disadvantage: Requires more storage than a sparse index.

Q7. Extendible Hashing with Directory Doubling

Extendible hashing is a dynamic hashing technique that uses a directory and buckets.

Important terms:

• Global depth – number of hash bits used by the directory.

• Local depth – number of bits used by a bucket.

• Bucket overflow causes splitting.

• If local depth equals global depth, the directory doubles.

Assume bucket capacity = 2 records.

Insert 8 records: 1, 2, 3, 4, 5, 6, 7, 8.

Initially, global depth = 1:

0 → Bucket A

1 → Bucket B

When overflow occurs, the corresponding bucket is split.

If local depth equals global depth, directory doubling occurs:

Global Depth 1 → DOUBLE → Global Depth 2

Directory:

00 → Bucket

01 → Bucket

10 → Bucket

11 → Bucket

If another bucket requires doubling:

Global Depth 2 → DOUBLE → Global Depth 3

Directory entries:

000, 001, 010, 011, 100, 101, 110, 111

Process:

Bucket overflow → Check local depth → Split if possible → Otherwise double directory → Split bucket → Reinsert records.

Conclusion: Extendible hashing grows dynamically and avoids the fixed bucket limitation of static hashing.

Q8. Secondary Storage Block Access – Sequential vs Indexed Retrieval
Assume:

• Total records = 1,000

• Records per block = 10

Number of blocks:

1000 / 10 = 100 blocks

Sequential Retrieval:

Worst case = 100 I/O operations

Average ≈ 50 I/O operations

Indexed Retrieval:

Suppose the index requires 2 blocks.

Index search = 2 I/Os

Data block access = 1 I/O

Total indexed cost:

2 + 1 = 3 I/O operations

Comparison:

Method	Worst-case I/O
Sequential	100
Indexed	3

Sequential access:

Block 1 → Block 2 → Block 3 → ... → Block 100 → Required Record

Indexed access:

Index → Required Block → Record

Conclusion: Indexed retrieval significantly reduces disk I/O compared with sequential scanning, especially for large files.

Q9. Sparse Index vs Dense Index with Timing Analysis

A dense index contains an entry for every record. A sparse index contains entries for selected records, usually one per block.

Assume:

• Records = 10,000

• Records per block = 100

• Blocks = 100

Dense index entries = 10,000

Sparse index entries = 100

C Program for simple timing comparison:

```
#include <stdio.h>
#include <time.h>
int main() {
    int n = 10000;
    int key = 9999;
    int a[10000];
    for (int i = 0; i < n; i++)
        a[i] = i;
    clock_t start, end;
    start = clock();
    for (int i = 0; i < n; i++) {
        if (a[i] == key)
            break;
    }
    end = clock();
    printf("Sequential search time: %f\n",
           (double)(end - start) / CLOCKS_PER_SEC);
    start = clock();
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (a[mid] == key)
            break;
```

```
        else if (a[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    end = clock();
    printf("Indexed/Binary search time: %f\n",
          (double)(end - start) / CLOCKS_PER_SEC);
    return 0;
}
```

Comparison:

Dense Index: every record, high storage, faster direct access.

Sparse Index: selected records, low storage, slightly more search work.

Conclusion: A dense index provides faster direct access but requires more storage. A sparse index saves storage and is effective for sorted files.

Q10. Construct and Traverse a B+ Tree and Perform Range Query 15–45
Consider the keys:

5, 10, 15, 20, 25, 30, 35, 40, 45, 50

A simplified B+ Tree:

```
            [20 | 35]
           /    |    \
    [5 10 15] [20 25 30] [35 40 45 50]
```

Leaf links:

[5 10 15] → [20 25 30] → [35 40 45 50]

Range query: 15 to 45

First search for 15. Then follow the leaf-level links:

15 → 20 → 25 → 30 → 35 → 40 → 45

Matching records:

15 20 25 30 35 40 45

Simple C Program:

```
#include <stdio.h>

int main() {
   int data[] = {
      5, 10, 15, 20, 25,
      30, 35, 40, 45, 50
   };
   int n = 10;
   int low = 15, high = 45;
   printf("Records from %d to %d:\n", low, high);
   for (int i = 0; i < n; i++) {
      if (data[i] >= low && data[i] <= high)
         printf("%d ", data[i]);
   }
   return 0;
}
```

Output:

Records from 15 to 45:

15 20 25 30 35 40 45

Conclusion: B+ Tree range queries are efficient because the tree locates the starting key and linked leaves provide sequential access to the remaining keys.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases